

4. Plantillas HTML y componentes web

Los **componentes web** son un conjunto de **tecnologías** que tienen como objetivo la **creación de elementos HTML personalizados** que puedan ser **reutilizados**. Su funcionalidad puede quedar **encapsulada** de manera que no afecte a otros componentes o al resto del documento. Dichas tecnologías son:

- **Custom Elements**. API para definir **elementos HTML personalizados**.
- **Shadow DOM**. DOM oculto y opcionalmente aislado del DOM del documento, con acceso a su propio espacio de nombres y estilos.
- **HTML templates**. Plantillas con **fragmentos de código HTML** que pueden ser almacenadas para ser insertadas en el documento desde JavaScript.

4.1. Custom Elements

Esta tecnología permite crear **elementos HTML** personalizados. Por ejemplo, podemos definir un elemento `<mi-contador>` que implemente un contador que se vaya incrementando cuando se haga clic sobre él.

! IMPORTANTE

Los nombres de los elementos personalizados **deben contener un guion** para diferenciarlos de los elementos estándar.

Para definir un elemento personalizado, hay que ejecutar el código siguiente:

```
customElements.define('nombre-elemento', ClaseElemento, { extends:
  'NombreElementoBase' });
```

El primer parámetro es el **nombre del elemento**, que debe incorporar **un guion**; el segundo, la **clase** que define el prototipo del elemento (comportamiento); por último, un objeto con la propiedad **extends**, que especifica el elemento del que hereda. El último parámetro solo es necesario si vamos a **personalizar un elemento estándar de HTML**, como un `<button>`; si se va a desarrollar un elemento personalizado completamente nuevo, no es necesario indicarlo.

Antes de definir el elemento, hay que **crear la clase** que se utilizará como prototipo.

🔗 EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A04_01.html`, con el código de ejemplo.

Podemos ver en él un ejemplo de clase que define la funcionalidad del elemento personalizado. Dicha clase puede definir los siguientes **métodos**:

- **constructor**. Función constructora. Dentro de ella hay que llamar a `super()`. Se ejecuta cuando **se crea** el componente. El componente todavía no está añadido al DOM, por lo que en su interior **no podemos hacer referencia a las funciones del DOM** (aunque sí del Shadow DOM, como veremos más adelante).
- **connectedCallback**. Función *callback* que se ejecuta cuando **se añade el componente al DOM**. Dentro de ella sí que tenemos **acceso a las funciones del DOM**.
- **disconnectedCallback**. Función *callback* que se ejecuta cuando **se elimina el componente del DOM**.
- **adoptedCallback**. Función *callback* que se ejecuta cuando **se mueve el componente a otro lugar del DOM**.
- **attributeChangedCallback**. Función *callback* que se ejecuta cuando **se modifica un atributo del componente**. Los atributos deben declararse en `observedAttributes`.

- `static get observedAttributes()`. Función que debe devolver un *array* con el listado de atributos que serán observables; y cuya actualización provocará, por tanto, una llamada a `attributeChangedCallback`.

Por simplicidad, en el ejemplo solo se muestra el funcionamiento de los dos primeros. Puedes consultar más información sobre el resto en los enlaces de la unidad.

4.2. Shadow DOM

En la sección anterior hemos visto cómo construir elementos personalizados formados por nodos del DOM. Sin embargo, el contenido de dichos elementos personalizados es **visible** dentro del documento: cualquier código JavaScript o regla de estilo CSS podría modificar la estructura interna de esos elementos. Para evitarlo, podemos utilizar el **Shadow DOM**.

El **Shadow DOM** es una característica que nos permite **aislar la estructura interna** de los elementos personalizados del resto del documento HTML. Su objetivo es favorecer el **encapsulamiento**.

! IMPORTANTE

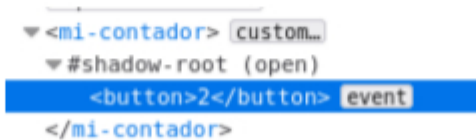
El Shadow DOM se utiliza para implementar la funcionalidad de controles HTML complejos, como, por ejemplo, el control `<input type="range">`.

Una de las principales ventajas que tiene el Shadow DOM es que permite definir **estilos CSS** que **no afectan al resto del documento**. Además, su contenido **no es accesible desde el exterior** por las funciones convencionales del DOM (`querySelector`, por ejemplo), sino que solo es accesible a través de la propiedad `shadowRoot` del elemento. Más adelante entraremos más en detalle.

Para utilizar esta característica, en nuestro ejemplo anterior modificaremos la función `connectedCallback`, para que quede definida de la siguiente manera:

```
connectedCallback() {  
    // Añadimos el Shadow DOM al elemento  
    const shadow = this.attachShadow({mode: 'open'});  
  
    let boton = document.createElement("button");  
    boton.textContent = this.cuenta;  
    boton.addEventListener("click", () => {  
        this.contar()  
        boton.textContent = this.cuenta;  
    });  
  
    // Añadimos el botón al Shadow DOM del elemento, en lugar de  
    // añadirlo al elemento directamente  
    shadow.append(boton);  
}
```

De esta manera, si inspeccionamos el código, veremos que el contenido del elemento está marcado como *shadow*:



```
<mi-contador> custom...  
  #shadow-root (open)  
    <button>2</button> event  
  </mi-contador>
```

Mediante `attachShadow` podemos activar el Shadow DOM en un elemento. Esta función devuelve el Shadow DOM, que también será accesible a través de la propiedad `shadowRoot` del elemento. Existen dos maneras de activar el Shadow DOM:

- `mode: 'open'`. El Shadow DOM es accesible a través de la propiedad `shadowRoot` del elemento.
- `mode: 'closed'`. El Shadow DOM no es accesible a través de la propiedad `shadowRoot` del elemento: `element.shadowRoot = null`.

Las funciones convencionales del DOM pueden ser utilizadas también dentro del Shadow DOM (`innerHTML`, `children`, `querySelector`, etcétera). Imaginemos la siguiente estructura pseudo-HTML:

```
<body>
  <p class="parrafo" id="p1">Párrafo 1</p>
  <p class="parrafo" id="p2">Párrafo 2</p>

  <mi-elemento id="elem1">
    <!-- Su contenido está creado en el Shadow DOM en modo 'open' -->
    <p class="parrafo" id="p1">Párrafo 3</p> <!-- Es válido: 'p1' está aislado
del DOM exterior -->
    <p class="parrafo" id="p4">Párrafo 4</p>
  </mi-elemento>
</body>
```

Como puede apreciarse, dentro del Shadow DOM se puede definir un `id` que esté utilizado fuera, dado que no colisionan. Veamos también qué devolverían las siguientes consultas:

```
document.getElementById("p1"); // Devuelve el primer párrafo
document.getElementById("p4"); // undefined (está en el Shadow DOM)

let miElemento = document.getElementById("elem1");
miElemento.shadowRoot.getElementById("p1"); // Devuelve el párrafo 3
(accedemos a través del Shadow DOM)

document.querySelectorAll("p"); // Devuelve los dos primeros párrafos
miElemento.shadowRoot.querySelectorAll("p"); // Devuelve los dos últimos
párrafos
```

Vemos que no es posible acceder al interior del Shadow DOM desde el exterior a través de métodos del DOM: la única manera de hacerlo es ejecutando las consultas a través de `shadowRoot`.

! IMPORTANTE

Solo puede crearse un Shadow DOM por elemento.

El Shadow DOM puede crearse en elementos personalizados y en determinadas etiquetas HTML estándar (no en todas). Puedes consultar los enlaces para obtener más información.

4.3. Templates

Las plantillas HTML son trozos de código HTML que solo se muestran cuando se insertan desde JavaScript. Se declaran dentro del elemento `<template>`. Permiten definir estructuras HTML complejas de manera declarativa utilizando exclusivamente HTML, sin necesidad de utilizar los métodos del DOM, como `createElement` o `append`. De esta manera, el diseño de componentes se hace más sencillo, ya que se puede desarrollar como una página convencional, y, cuando esté lista, copiarla dentro de una etiqueta `<template>` para su utilización.

A continuación, puedes ver un ejemplo de plantilla HTML:

```
<template id="mi-boton">
  <button>Mi Botón</button>
</template>
```

Si copias el código anterior en una página HTML y lo cargas, será ignorado por el navegador. Solo puede ser insertado mediante JavaScript de la siguiente manera:

```
let plantilla = document.getElementById('mi-boton'); // Elemento <template>
let plantillaContenido = plantilla.content; // Contenido de la plantilla
(nodo tipo 'DocumentFragment'
document.body.append(plantillaContenido.cloneNode(true)); // Se añade a
<body> una COPIA de la plantilla
document.body.append(plantillaContenido.cloneNode(true)); // Se añade a
<body> una segunda COPIA de la plantilla
```

En el código anterior, `plantilla` hace referencia al elemento `<template>`. La propiedad `content` de un elemento `<template>` hace referencia al contenido de la plantilla, que es un tipo especial de nodo del DOM llamado `DocumentFragment`. Un nodo de tipo `DocumentFragment` se comporta como un trozo de documento que no está insertado en el DOM. Aun así, en él se pueden utilizar los métodos del DOM, como, por ejemplo, `querySelector` o `children`, para hacer búsquedas o modificaciones, o navegar por él.

Es posible utilizar estilos dentro de las plantillas mediante etiquetas `<style>`. Sin embargo, dichos estilos tendrán alcance global dentro del documento. Si queremos que esos estilos estén aislados y solo afecten a los elementos de la plantilla, tendremos que utilizar la plantilla junto con el Shadow DOM.

A continuación, podemos ver nuestro ejemplo de contador utilizando elementos personalizados, Shadow DOM y plantillas.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A04_02.html`, con el código de ejemplo.

4.4. Slots

Nuestro componente `<mi-contador>` ya es totalmente funcional: tiene una etiqueta asociada, está correctamente encapsulado mediante el Shadow DOM y utiliza una plantilla HTML para definir su estructura y sus estilos, evitando así tener que definir una estructura HTML compleja mediante los métodos del DOM, como `createElement` o `append`.

Sin embargo, todavía le falta un detalle: no admite etiquetas internas, como cualquier otra etiqueta HTML. Por ejemplo, no podemos hacer lo siguiente:

```
<mi-contador>
  <span>Texto personalizado</span>
</mi-contador>
```

Esta funcionalidad puede ser añadida mediante elementos `<slot>`. Esos elementos se definen en la plantilla y pueden ser invocados en la utilización de los componentes personalizados mediante atributos. La declaración en la plantilla se realizaría de la siguiente manera:

```
<!-- Suponiendo que se instancia en un Componente personalizado
'mi-componente' -->
<template>
  <div>
    <slot name="texto">Texto por defecto</slot>
  </div>
</template>
```

Y su utilización sería así:

```
<mi-componente>
  <span slot="texto">Juan</span>
</mi-componente>

<mi-componente>
  <p slot="texto">Laura</p>
</mi-componente>

<mi-componente>
</mi-componente>
```

El resultado que aparecería en el navegador sería:

```
<div>
  <slot name="titulo">
    <span slot="texto">Juan</span>
  </slot>
</div>

<div>
  <slot name="titulo">
    <p slot="texto">Laura</p>
  </slot>
</div>

<div>
  <slot name="titulo">
    Texto por defecto
  </slot>
</div>
```

De esta manera, el código del componente **mi-contador** completo, con slots personalizados, quedaría de la siguiente manera.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A04_03.html**, con el código de ejemplo.

4.5. Importación de componentes

La utilización de componentes web implica:

- Crear plantillas en formato HTML.
- Utilizar JavaScript para crear las clases de los componentes personalizados, inicializar el Shadow DOM e importar el código HTML de dichas plantillas.

Cuando el número de componentes que hay que manejar es lo suficientemente grande, es conveniente **dividir el código en varios archivos**. Con los ficheros JavaScript no hay mucho problema, dado que podemos utilizar el sistema de **módulos** mediante **import**; sin embargo, actualmente **no hay ningún mecanismo estándar para importar archivos HTML**.

Idealmente, sería muy interesante disponer de este mecanismo, ya que permitiría trabajar con ficheros HTML para diseñar las plantillas de manera independiente. Dado que no es posible, hay **tres enfoques** fundamentales para solucionar este problema:

- Utilizar **fetch** para descargar el archivo de plantilla HTML y almacenar su contenido en una variable. Veremos el uso de **fetch** en unidades posteriores.
- Una vez diseñada la plantilla en un archivo HTML, coger su código y copiarlo en una **variable** de tipo **cadena de acento grave**, `` ``, que se **exporte** dentro de un **archivo de módulo de JavaScript**. El archivo JavaScript puede ser cargado mediante **import** y así **acceder a la variable exportada que almacene la cadena de texto con la plantilla**.
- Utilizar alguna herramienta de empaquetado, como **Webpack**, que veremos en unidades posteriores.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, dispones de un enlace al recurso *¿Qué pasa con import y los web components?*, en el que se trata más a fondo este tema.

4.6. Comparación con otros sistemas de plantillas

Aparte de las plantillas HTML, existen otros sistemas de plantillas que pueden ser utilizados en HTML. Hay dos grandes categorías:

- Plantillas utilizadas con lenguajes de **servidor**. Incorporan trozos de HTML junto con **directivas** propias del lenguaje de servidor utilizado (PHP, Java, etcétera). Normalmente se utilizan en arquitectura de tipo **MVC**. Cuando se sirve la página al cliente, el servidor compila la plantilla y genera una **página HTML** totalmente compatible con el navegador. Las **directivas** PHP o Java utilizadas **no llegan nunca al navegador**.
- Plantillas utilizadas en **entorno cliente**. Utilizan **librerías JavaScript**. Incorporan trozos de HTML junto con **directivas** de la librería utilizada. En este caso, hay dos posibilidades:
 - Es necesario un proceso de **conversión a HTML** para que la librería compile las directivas y produzca un documento HTML compatible con el navegador. Normalmente, esta compilación se realiza desde algún entorno de ejecución, como Ruby o NodeJS. Los **gestores de contenidos estáticos**, como **Jekyll** (utilizado en GitHub) o **Hugo**, utilizan un sistema de plantillas que son compiladas para producir un conjunto de archivos HTML estándar.
 - La plantilla incluye **código JavaScript**, que realiza un proceso de **compilación** en cuanto se carga la página. La compilación busca las directivas de la página y **genera HTML válido** a partir de ellas. Las librerías como **Angular**, **VueJS** o **React** utilizan este enfoque. En este caso, el navegador recibe tanto el código **HTML** como las **directivas** del sistema de plantillas.

Algunos de los sistemas de plantillas basados en librerías más utilizados son:

- **Mustache**
- **Handlebars**
- **pug**
- **ejs**